

RAPPORT TECHNIQUE ET ACADÉMIQUE

Freelancer Hub

Plateforme de mise en relation entre clients et freelancers

Architecture applicative basée sur **Ionic/Angular** et **Flask**

Nature du document : Rapport technique

Type de solution : Application web/mobile de services freelance

Groupe : Ameni Tlili, Ameni Chakhari, Yasmine Bouchami, Sirine Zaibi

Date : 5 mai 2026

Table des matières

1	Introduction générale	5
2	Contexte et objectifs du projet	5
2.1	Contexte	5
2.2	Objectifs fonctionnels	5
2.3	Objectifs techniques	6
3	Présentation générale de la solution	6
4	Architecture technique de l'application	7
4.1	Architecture logique	7
4.2	Organisation du backend	8
4.3	Organisation du frontend	9
5	Guide de lecture technique du code	11
5.1	Fichiers d'entrée et d'initialisation du backend	11
5.2	Organisation interne du backend	11
5.3	Fichiers clés côté frontend	12
5.4	Cartographie technique des routes backend	12
5.5	Services métiers importants à connaître	14
5.6	Collections MongoDB et leur utilité	14
5.7	Points de vigilance pour les développeurs	14
6	Structure détaillée du frontend Ionic/Angular	15
6.1	Point d'entrée frontend	15
6.2	Routage	15
6.3	Pages	15
6.4	Services Angular	16
6.5	Guards	16

6.6 Composants	16
7 Liaison frontend – backend	16
7.1 Base URL	17
7.2 Principe des appels	17
7.3 Exemples de correspondances entre frontend et backend	17
8 Authentification et sécurité	18
8.1 JWT	18
8.2 Vérification du rôle	18
8.3 CORS	19
8.4 Sensibilité des secrets	19
9 Base de données MongoDB	19
10 Flux métiers principaux	19
10.1 Inscription et login	20
10.2 Rechercher et engager un freelancer	20
10.3 Exécution d’une commande gig	20
10.4 Travail par contrat et sprints	20
10.5 Chat	20
11 Analyse fonctionnelle	20
11.1 Fonctionnalités de base côté client	20
11.1.1 Déposer un job	21
11.1.2 Rechercher un profil freelancer	21
11.1.3 Communiquer avec un freelancer via message	21
11.1.4 Effectuer le paiement	21
11.2 Fonctionnalités de base côté freelancer	21
11.2.1 Postuler à un job	22
11.2.2 Publier ses gigs	22

11.2.3	Suivre l'exécution de ses missions	22
11.3	Explication détaillée des routes backend	22
11.3.1	Routes d'authentification	22
11.3.2	Routes liées aux freelancers	23
11.3.3	Routes liées aux clients	23
11.3.4	Routes de gestion des gigs et du catalogue	23
11.3.5	Routes de jobs et de propositions	23
11.3.6	Routes de commandes	24
11.3.7	Routes de contrats, plans de sprint et sprints	24
11.3.8	Routes de paiement	24
11.3.9	Routes de chat et de communication temps réel	24
11.3.10	Routes d'administration	25
12	Architecture applicative détaillée	25
12.1	Couche backend	25
12.2	Couche frontend	25
12.3	Données manipulées par le système	26
13	Gestion des projets en sprints	26
13.1	Justification du découpage en sprints	26
13.2	Cycle de vie d'un sprint fonctionnel	26
13.3	Bénéfices attendus	27
14	Planification du développement en sprints	27
15	Exigences non fonctionnelles	28
15.1	Sécurité	28
15.2	Performance	28
15.3	Maintenabilité et évolutivité	28
15.4	Ergonomie et expérience utilisateur	28

16 Discussion et appréciation technique	29
17 Conclusion générale	29

1 Introduction générale

La transformation numérique des activités de services a favorisé l'émergence de plateformes capables de mettre en relation des porteurs de besoins et des prestataires spécialisés. Dans ce contexte, **Freelancer Hub** est conçu comme une application permettant à un client de publier un besoin, d'identifier un profil pertinent, d'échanger avec le freelancer sélectionné, puis de contractualiser et payer la prestation dans un environnement structuré.

Sur le plan technique, la solution s'appuie sur un frontend **Ionic** + **Angular** destiné à fournir une interface moderne, réactive et compatible avec plusieurs supports, ainsi que sur un backend **Flask** chargé d'exposer les services métiers, d'assurer les contrôles de sécurité et de gérer les flux de données. Le système intègre également des mécanismes de messagerie et un découpage des projets en sprints afin de suivre l'avancement de manière progressive.

L'objectif du présent rapport est de proposer une présentation claire, rigoureuse et académique de la solution, en détaillant son architecture, ses fonctionnalités principales, son organisation technique ainsi que son mode de réalisation incrémental.

2 Contexte et objectifs du projet

2.1 Contexte

Les plateformes de freelancing répondent à un double besoin : d'une part, elles permettent aux clients de trouver rapidement des compétences ciblées ; d'autre part, elles offrent aux freelancers un espace de visibilité, de prospection et de gestion de leurs missions. Toutefois, la simple mise en relation ne suffit pas. Une plateforme moderne doit aussi intégrer la communication, le suivi de projet, la traçabilité des échanges et la sécurisation du paiement.

Freelancer Hub s'inscrit dans cette logique en proposant une solution intégrée, orientée vers la fluidité de l'expérience utilisateur et la structuration du cycle de vie d'une mission.

2.2 Objectifs fonctionnels

Le projet poursuit les objectifs suivants :

- permettre à un client de déposer un job décrivant précisément son besoin ;
- offrir un moteur de recherche facilitant l'identification de freelancers qualifiés ;
- intégrer un système de messagerie pour centraliser les échanges ;
- prendre en charge le paiement des prestations dans un flux sécurisé ;
- permettre au freelancer de postuler à des jobs et de publier des gigs ;

- organiser la conduite des projets en plusieurs sprints afin de mieux piloter les livraisons.

2.3 Objectifs techniques

Sur le plan technique, la solution vise à :

- séparer clairement la couche présentation et la couche métier ;
- proposer une architecture modulaire et maintenable ;
- sécuriser les accès via authentification et contrôle des rôles ;
- rendre possible une évolution progressive du produit ;
- garantir la réutilisation des composants côté frontend et backend.

3 Présentation générale de la solution

Freelancer Hub repose sur une architecture client-serveur dans laquelle l'application frontend dialogue avec une API backend. Le frontend, développé avec Ionic et Angular, prend en charge les écrans, la navigation, les formulaires, les appels réseau et la gestion locale du contexte utilisateur. Le backend, développé avec Flask, expose les endpoints métiers, orchestre la logique applicative, applique les règles de sécurité et dialogue avec la base de données.

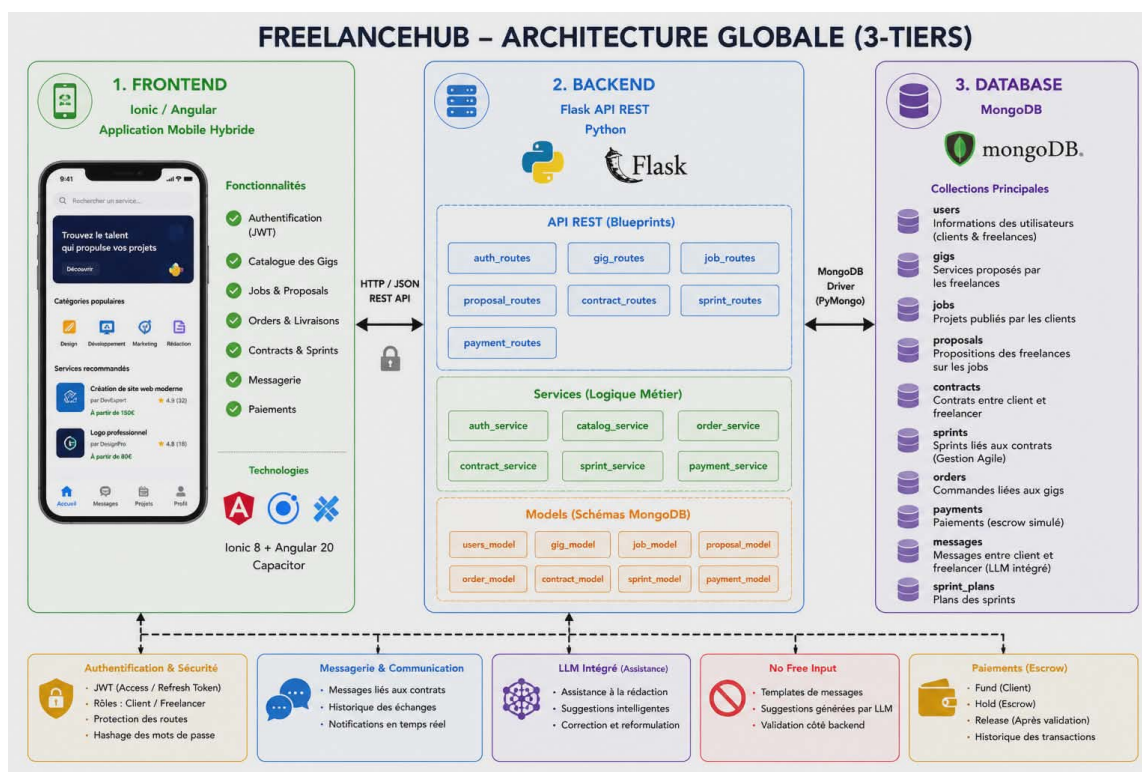


FIGURE 1 – Vue d'ensemble de l'architecture globale de Freelancer Hub en trois tiers : frontend, backend et base de données.

La figure 1 illustre de manière synthétique l'architecture générale de la plateforme. Elle met en évidence la séparation en trois niveaux : le frontend Ionic/Angular chargé de l'expérience utilisateur, le backend Flask structuré autour de blueprints et de services métiers, et la base de données MongoDB destinée à la persistance des entités principales telles que les utilisateurs, gigs, jobs, propositions, contrats, paiements et plans de sprint.

Le fonctionnement global suit la séquence suivante :

1. l'utilisateur interagit avec une page de l'application ;
2. la page appelle un service Angular ;
3. le service envoie une requête HTTP vers l'API Flask ;
4. Flask transmet la requête à une route métier ;
5. la route appelle un service Python ;
6. le service traite la logique métier et l'accès aux données ;
7. une réponse JSON est renvoyée au frontend ;
8. l'interface met à jour l'affichage et l'état de l'application.

4 Architecture technique de l'application

4.1 Architecture logique

L'architecture logique est organisée autour de quatre blocs :

- **couche présentation** : composants, pages, formulaires et navigation Angular/Ionic ;
- **couche services frontend** : encapsulation des appels API et mutualisation des traitements côté client ;
- **couche métier backend** : routes Flask, services Python, contrôle des rôles et règles de validation ;
- **couche données** : persistance des comptes, profils, jobs, propositions, messages, paiements et sprints.

Cette séparation favorise la maintenabilité du code, la lisibilité des responsabilités et la possibilité de faire évoluer les modules sans remettre en cause l'ensemble de l'application.

4.2 Organisation du backend

Le backend s'articule autour d'une structure modulaire qui distingue les routes, les services, les modèles et les composants temps réel. Les captures fournies mettent en évidence une organisation en plusieurs sous-dossiers spécialisés.

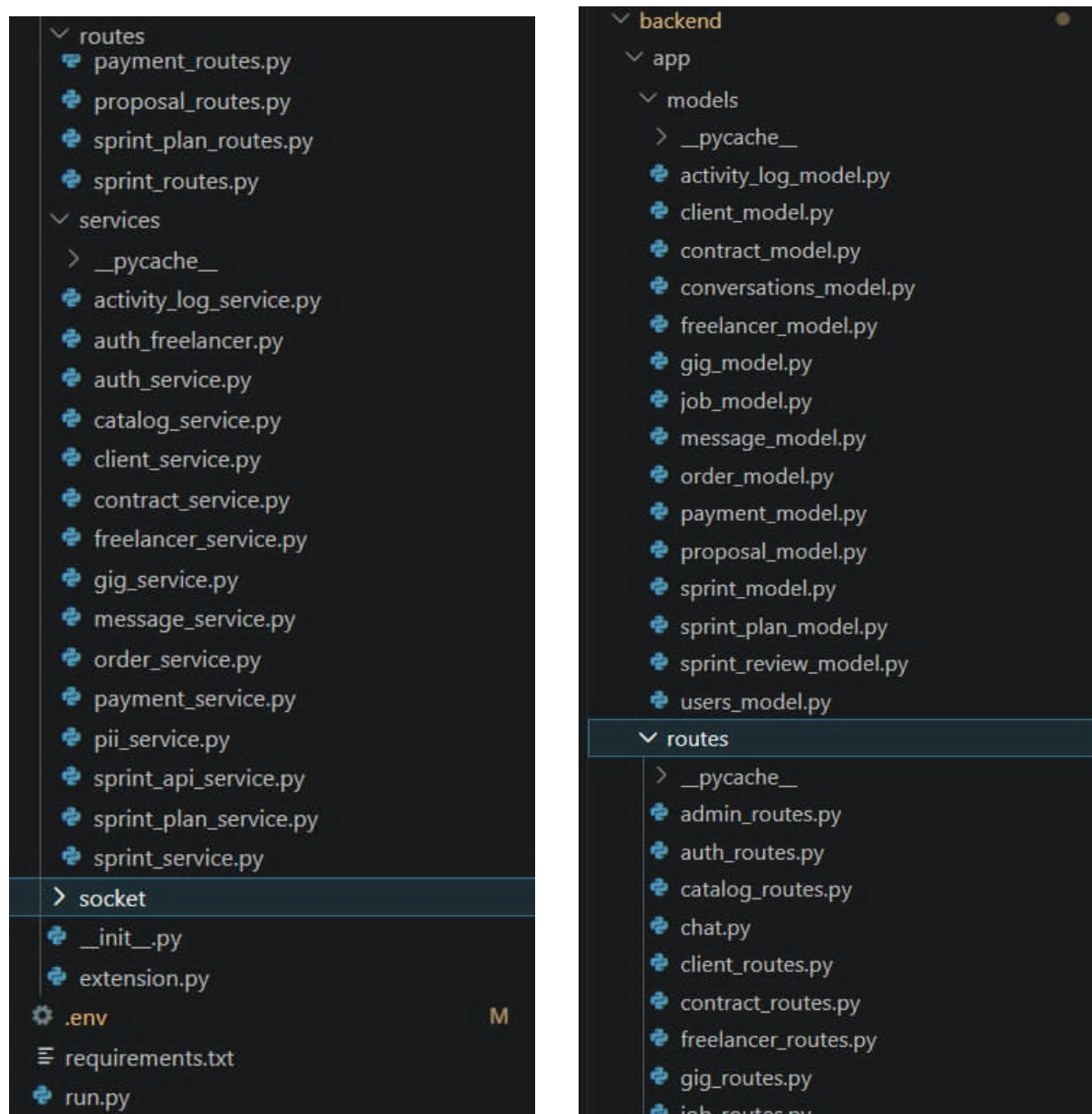


FIGURE 2 – Organisation du backend : routes, services, modèles et composants applicatifs.

À partir de cette structure, on distingue les éléments suivants :

- **Routes** : elles exposent les endpoints HTTP et constituent la porte d'entrée des actions métiers ;
- **Services** : ils concentrent la logique métier, les contrôles fonctionnels et l'orchestration des opérations ;

- **Models** : ils servent à structurer et centraliser l'accès aux collections de données ;
- **Socket** : ce module prend en charge les interactions temps réel, notamment pour la messagerie.

4.3 Organisation du frontend

Le frontend est structuré autour des pages, composants, services, guards et environnements. Les captures montrent un découpage riche en écrans dédiés aux profils, aux jobs, aux gigs, aux commandes et aux interactions conversationnelles.

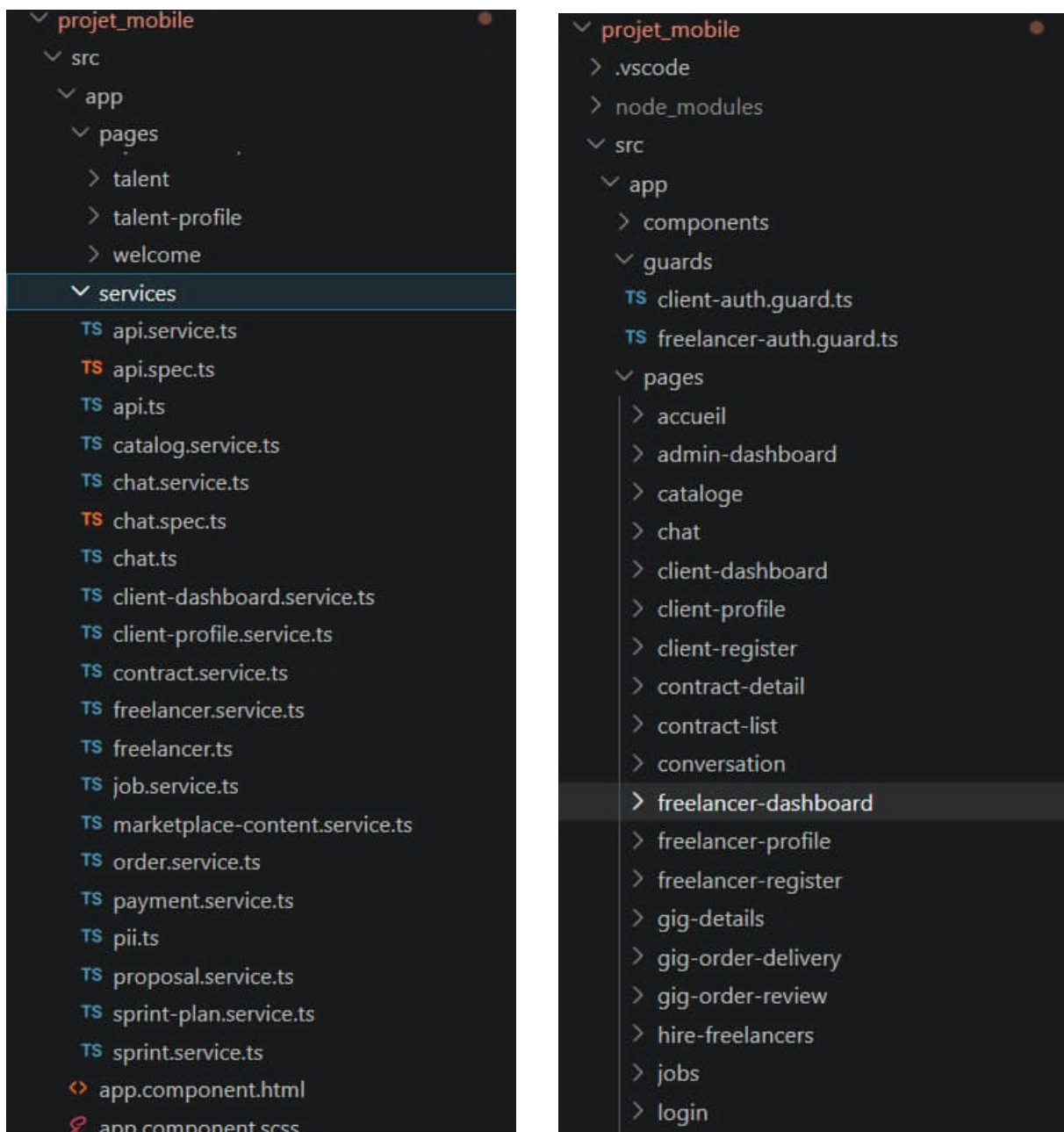


FIGURE 3 – Organisation du frontend Ionic/Angular : services, pages et modules d'interface.

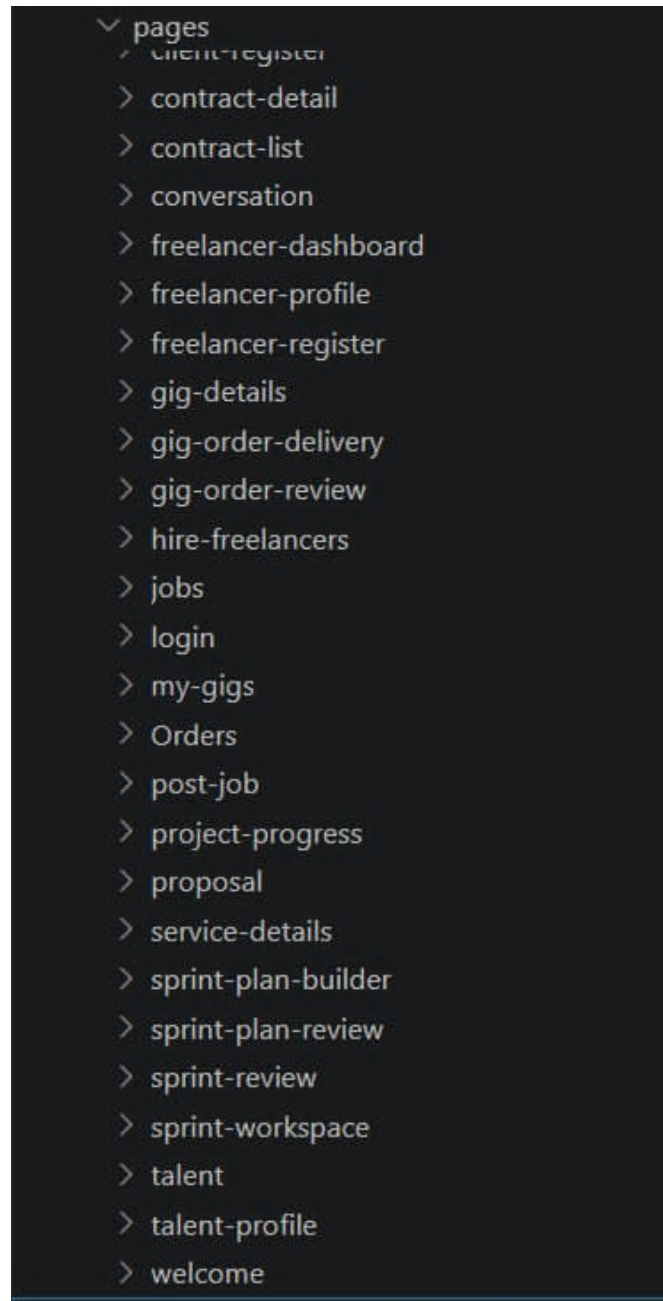


FIGURE 4 – Extrait de l'arborescence des pages de l'application.

Cette structuration permet :

- de séparer les écrans métiers selon le rôle utilisateur ;
- de centraliser les appels API dans des services réutilisables ;
- d'isoler les mécanismes de protection d'accès via les guards ;
- de maintenir une cohérence d'interface sur l'ensemble des parcours.

5 Guide de lecture technique du code

Cette section a pour objectif d'aider un développeur à comprendre rapidement où se trouvent les responsabilités majeures dans le code, comment les fonctionnalités sont réparties et dans quel ordre il est pertinent de lire les fichiers.

5.1 Fichiers d'entrée et d'initialisation du backend

Fichier `run.py`

`backend/run.py` constitue le point d'exécution de l'application côté serveur. Son rôle est de créer l'instance Flask puis de lancer l'application, généralement via `socketio.run(app, debug=True)`. Pour un développeur, ce fichier est utile pour comprendre comment l'application démarre réellement, dans quel mode elle s'exécute et par quel mécanisme le temps réel est activé.

Fichier d'initialisation

`backend/app/__init__.py` est le cœur de l'assemblage applicatif. C'est dans ce fichier que s'opèrent le chargement de la configuration, l'initialisation des extensions, la connexion à MongoDB, la configuration de JWT et l'enregistrement des blueprints. En pratique, ce fichier explique comment toutes les briques du backend sont reliées entre elles.

Fichier `extension.py`

`backend/app/extension.py` centralise les objets partagés, notamment `socketio`, `db`, `jwt` et `bcrypt`. Cette centralisation évite d'instancier plusieurs fois les mêmes composants dans des modules différents et facilite leur réutilisation. Pour un nouveau développeur, ce fichier sert à identifier les dépendances globales du backend.

5.2 Organisation interne du backend

Le backend suit une structure en couches qui facilite la compréhension du flux d'une requête.

Pattern d'exécution backend

1. une requête HTTP arrive sur une route Flask ;
2. la route lit les données de `request` et vérifie le rôle utilisateur ;
3. la route appelle un service métier ;
4. le service applique la logique fonctionnelle ;
5. le service lit ou modifie des données dans MongoDB ;
6. une réponse JSON et un code HTTP sont renvoyés au frontend.

Routes, services, modèles et socket

Routes : elles exposent les endpoints REST et découpent le projet par domaine fonctionnel.

Services : ils portent la logique métier réelle : validations, règles de transition, orchestration des opérations.

Models : ils structurent l'accès aux collections MongoDB sans constituer un ORM classique.

Socket : il ajoute la communication temps réel, surtout pour la messagerie.

5.3 Fichiers clés côté frontend

Fichier `app.routes.ts`

`projet_mobile/src/app/app.routes.ts` définit les routes de navigation Angular. Ce fichier permet de comprendre quels écrans existent, comment les parcours utilisateur sont structurés et où interviennent les guards d'accès selon le rôle connecté.

Fichier `api.service.ts`

`projet_mobile/src/app/services/api.service.ts` joue le rôle de service central d'appel réseau. Il sert souvent de base commune aux requêtes HTTP vers l'API. Pour un développeur, ce fichier est important car il montre comment l'URL backend est utilisée, comment les appels sont factorisés et comment les réponses sont consommées.

Services frontend spécialisés

`freelancer.service.ts`, ainsi que les autres services spécialisés, encapsulent les opérations liées à un domaine précis : profils, gigs, commandes, paiements, chat ou sprints. Cette spécialisation améliore la lisibilité fonctionnelle mais demande une convention claire pour éviter le chevauchement avec un service API central.

Fichier `environment.ts`

`projet_mobile/src/environments/environment.ts` contient la configuration d'environnement, notamment l'URL de l'API. Ce fichier est stratégique car il évite de disperser des URLs codées en dur dans les services frontend et facilite le passage entre développement, recette et production.

5.4 Cartographie technique des routes backend

Afin de rendre la lecture plus opérationnelle, les routes peuvent être regroupées dans des encadrés par domaine métier.

Authentification

`auth_routes.py` gère l'inscription et la connexion. Les endpoints majeurs sont `POST /api/auth/register/client`, `POST /api/auth/register/freelancer` et `POST /api/auth/login`. Ces routes créent les comptes, vérifient les identifiants et génèrent le JWT contenant l'identité et le rôle de l'utilisateur. Elles sont le point de départ de tout

accès sécurisé au reste du système.

Profils freelancers

`freelancer_routes.py` couvre la lecture du profil, sa mise à jour, le changement de mot de passe, l'accès au dashboard freelancer et les fonctionnalités de recherche publique. Une partie des routes est publique afin de favoriser la découverte de talents, tandis que les opérations de gestion nécessitent un token JWT et le rôle **freelancer**.

Profils clients

`client_routes.py` pilote le profil client, sa mise à jour et l'accès au dashboard client. Toutes ces opérations sont protégées, car elles manipulent des données personnelles et le pilotage direct des projets publiés par le client.

Gigs et catalogue

`gig_routes.py` est orienté gestion interne des services proposés par le freelancer : création, édition, suppression et lecture de ses gigs. En parallèle, les routes du catalogue sont orientées consultation côté client ou public : listing, filtres, détails et commande. Cette séparation améliore la clarté entre administration de l'offre et parcours d'achat.

Jobs et proposals

`job_routes.py` permet au client de publier et gérer ses besoins. `proposal_routes.py` permet au freelancer de répondre à ces besoins. Ensemble, ces deux fichiers implémentent le cycle : publication d'un besoin, réception de candidatures, comparaison des propositions, puis sélection éventuelle d'un freelancer en vue d'un contrat.

Orders

`order_routes.py` gère le cycle d'une commande issue d'un gig. Les états **pending**, **in_progress**, **submitted** et **completed** traduisent la progression du travail. Ce module est essentiel pour comprendre la logique de livraison et de validation dans le projet.

Contrats et sprints

`contract_routes.py`, `sprint_plan_routes.py` et `sprint_routes.py` structurent l'exécution avancée des missions. Ils permettent de formaliser l'accord, de planifier les itérations, de soumettre des livrables et d'organiser la validation progressive des travaux.

Paielement

`payment_routes.py` relie la progression métier à la progression financière. Les opérations de financement, de suivi et de déblocage des paiements sont sensibles et doivent être lues en parallèle des commandes, contrats et sprints pour comprendre la chaîne complète de valeur.

Chat et temps réel

`chat.py`, `chat_socket.py` et `message_service.py` couvrent la création des conversations, la récupération des messages et l'envoi en temps réel. Ce bloc technique est central pour comprendre comment la plateforme conserve l'historique des échanges tout en offrant une

interaction instantanée.

Administration

`admin_routes.py` regroupe les opérations de supervision : approbation, rejet, filtrage, statistiques et gestion de l'état des comptes. Ces routes donnent une vision transverse de la plateforme et sont particulièrement utiles pour comprendre les mécanismes de modération et de gouvernance du système.

5.5 Services métiers importants à connaître

Au-delà des routes, certains services doivent être lus en priorité, car ce sont eux qui contiennent les règles fonctionnelles principales.

Services backend prioritaires

`auth_service.py` et `auth_freelancer.py` pour comprendre l'inscription et l'authentification ;
`freelancer_service.py` et `client_service.py` pour les profils et dashboards ;
`gig_service.py`, `catalog_service.py` et `order_service.py` pour les gigs et commandes ;
`contract_service.py`, `sprint_plan_service.py`, `sprint_service.py` et `payment_service.py` pour les missions structurées et les paiements ;
`message_service.py` et `pii_service.py` pour la communication et la validation éventuelle des messages.

5.6 Collections MongoDB et leur utilité

Collections MongoDB principales

`users` stocke les comptes et rôles ; `clients` et `freelancers` stockent les profils métier ; `gigs`, `jobs`, `proposals` et `orders` matérialisent la relation commerciale ; `contracts`, `sprint_plans`, `sprints` et `payments` supportent le suivi contractuel et financier ; `conversations` et `messages` assurent la persistance des échanges.

5.7 Points de vigilance pour les développeurs

Vigilances techniques

- l'accès MongoDB n'est pas totalement unifié entre `current_app.db` et `app.extension.db` ;
- l'initialisation de `JWTManager()` semble exister à la fois dans `__init__.py` et `extension.py` ;
- le frontend combine un service API central et plusieurs services spécialisés, ce qui impose une convention claire ;

- certaines URLs peuvent être codées en dur alors qu'elles devraient être centralisées dans `environment.ts` ;
- certains fichiers présentent des problèmes d'encodage, avec un impact direct sur la lisibilité et la maintenance.

6 Structure détaillée du frontend Ionic/Angular

Cette partie complète la description générale déjà fournie en détaillant la manière dont le frontend est organisé pour supporter les parcours client et freelancer. L'objectif est d'aider un développeur à localiser rapidement les écrans, services et protections d'accès associés à chaque fonctionnalité.

6.1 Point d'entrée frontend

Le point d'entrée fonctionnel du frontend se situe dans le dossier `projet_mobile/src/app/`. Ce répertoire regroupe l'arborescence applicative Angular : pages, composants, guards, services et fichiers de routage. Il constitue le point de départ naturel pour comprendre l'expérience utilisateur implémentée côté client.

Structure principale du frontend

`pages/` contient les écrans métiers ; `services/` contient les appels API et la logique transverse côté client ; `guards/` protège les routes selon l'authentification et le rôle ; `components/` regroupe les éléments d'interface réutilisables ; les fichiers de routes structurent la navigation globale de l'application.

6.2 Routage

Le fichier `app.routes.ts` définit les chemins de navigation vers les différentes pages. Il permet de comprendre quels parcours sont accessibles aux visiteurs, aux clients connectés et aux freelancers connectés. Dans un projet Ionic/Angular, ce fichier est essentiel car il relie l'URL ou l'état de navigation à un composant de page précis.

Le routage joue aussi un rôle important dans l'application des guards : une route sensible peut être rendue accessible uniquement à un utilisateur authentifié, voire uniquement à un utilisateur dont le rôle correspond à l'espace visé.

6.3 Pages

Le dossier `pages/` regroupe les écrans visibles par l'utilisateur. D'après le dossier technique, on y retrouve notamment des pages liées à l'accueil, au catalogue, au chat, aux dashboards, aux profils, aux gigs, aux jobs, aux propositions, aux contrats et au suivi de projet.

Exemples de pages identifiées

accueil, admin-dashboard, catalog, chat, client-dashboard, client-profile, contract-detail, contract-list, conversation, freelancer-dashboard, freelancer-profile, gig-details, gig-order-delivery, gig-order-review, hire-freelancers, jobs, my-gigs, orders, post-job, project-progress, proposal, service-details, sprint-plan-builder, sprint-plan-review, sprint-review, sprint-workspace, talent et talent-profile.

Cette granularité montre que le frontend n'est pas limité à une interface simplifiée : il structure des parcours variés pour explorer les profils, publier un besoin, commander un gig, suivre une livraison, discuter avec un prestataire et piloter l'avancement d'une mission.

6.4 Services Angular

Le dossier `services/` constitue la passerelle fonctionnelle entre l'interface et le backend. Chaque service encapsule des appels HTTP vers un domaine métier donné. Le dossier technique identifie notamment `api.service.ts`, `catalog.service.ts`, `chat.service.ts`, `client-dashboard.service.ts`, `client-profile.service.ts`, `contract.service.ts`, `freelancer.service.ts`, `jobs.service.ts`, `orders.service.ts`, `payment.service.ts`, `proposal.service.ts`, `sprint-plan.service.ts` et `sprint.service.ts`.

Un développeur doit lire ces services pour comprendre quelles données sont envoyées au backend, quels endpoints sont consommés et comment les réponses sont exploitées par l'interface.

6.5 Guards

Les guards Angular assurent la protection des parcours. Ils vérifient généralement la présence d'un token et parfois le rôle de l'utilisateur. Le dossier mentionne notamment des guards de type client et freelancer. Leur rôle est d'empêcher l'accès à certaines pages lorsque l'utilisateur n'est pas authentifié ou ne correspond pas au rôle attendu.

6.6 Composants

Les composants réutilisables permettent d'uniformiser l'interface et d'éviter la duplication du code visuel. Ils peuvent porter des éléments comme des cartes de service, des listes de résultats, des formulaires, des blocs de navigation ou des zones de messagerie. Leur présence améliore la maintenabilité du frontend et la cohérence de l'expérience utilisateur.

7 Liaison frontend – backend

La valeur technique du projet repose en grande partie sur la qualité de la liaison entre l'interface Ionic/Angular et l'API Flask. Cette liaison détermine comment les actions utilisateur deviennent

des requêtes métier exploitables par le backend.

7.1 Base URL

Le dossier technique souligne l'importance de centraliser l'adresse de l'API dans `environment.ts`. Cette approche évite la dispersion de chaînes comme `http://localhost:5000/...` dans plusieurs services frontend et simplifie le changement d'environnement entre développement, test et production.

7.2 Principe des appels

Le principe général est le suivant : une page Angular appelle un service spécialisé ; le service construit la requête HTTP ; l'URL cible correspond à une route Flask ; le backend traite la requête et retourne un JSON ; le frontend met à jour l'interface à partir de cette réponse.

Chaîne de liaison frontend – backend

Page Angular
rightarrow service Angular
rightarrow requête HTTP
rightarrow blueprint Flask
rightarrow service Python
rightarrow MongoDB
rightarrow réponse JSON
rightarrow mise à jour de l'interface.

7.3 Exemples de correspondances entre frontend et backend

Authentification

Le frontend envoie une requête vers `POST /api/auth/login`. Côté backend, cette requête est prise en charge par `auth_routes.py`. Le backend vérifie l'email et le mot de passe, puis retourne un `access_token`, le `role` et le `user_id`. Le frontend stocke ensuite ce token pour les appels protégés.

Dashboard freelancer

Une page de tableau de bord freelancer consomme généralement `GET /api/freelancer/dashboard`. La route correspondante se trouve dans `freelancer_routes.py`. Le service backend renvoie les données nécessaires à l'affichage de l'activité, des commandes ou de la situation du compte freelancer.

Profil freelancer

Le frontend peut interroger `GET /api/freelancer/profile` afin d'afficher ou de mettre à jour le profil du prestataire. Cette route relie directement l'espace utilisateur aux informations professionnelles nécessaires à la mise en relation.

Recherche des talents

La recherche publique des profils est liée à des routes du domaine `freelancer`, par exemple `GET /api/freelancer`. Le frontend transmet les filtres saisis par l'utilisateur et affiche la liste des résultats renvoyés par le backend.

Commande d'un gig

Lorsqu'un client commande un service packagé, le frontend déclenche par exemple `POST /api/catalog/<gig_id>/order`. Cette action mène à la création d'une `order` côté backend et alimente ensuite le suivi de la commande dans les espaces client et freelancer.

Récupération des commandes client et freelancer

Le frontend consomme des routes comme `GET /api/orders/client` et `GET /api/orders/freelancer` pour afficher les commandes selon le rôle connecté. Les réponses sont ensuite utilisées pour construire les listes, filtres et vues détaillées dans les dashboards.

Chat

Les écrans de chat interagissent avec des routes sous `/api/chat/...` pour charger les conversations et l'historique, puis s'appuient sur Socket.IO pour les messages temps réel. Cette combinaison permet de conserver les échanges en base tout en garantissant une interaction immédiate dans l'interface.

8 Authentification et sécurité

8.1 JWT

Le mécanisme d'authentification repose sur des JSON Web Tokens. Après connexion, le backend génère un token qui porte l'identité de l'utilisateur et son rôle. Ce token est ensuite transmis dans les appels protégés afin que le backend puisse autoriser ou refuser l'accès aux ressources demandées.

8.2 Vérification du rôle

Le dossier technique souligne que la sécurité ne repose pas uniquement sur la présence d'un token, mais aussi sur la vérification du rôle. Certaines routes sont réservées au rôle `client`, d'autres au rôle `freelancer`, d'autres enfin au rôle `admin`. Cette séparation évite qu'un utilisateur accède à des fonctions qui ne relèvent pas de son espace métier.

8.3 CORS

Comme le frontend et le backend peuvent être servis séparément, la configuration CORS est nécessaire pour autoriser les échanges entre domaines ou ports différents. Elle doit être définie de manière suffisamment stricte pour éviter les ouvertures excessives tout en permettant le bon fonctionnement du frontend Ionic/Angular.

8.4 Sensibilité des secrets

Le dossier insiste sur la gestion prudente des secrets, notamment `JWT_SECRET_KEY` et `MONGO_URI`. Ces variables doivent être stockées hors du dépôt, idéalement dans un fichier `.env` non versionné ou dans une solution dédiée à la gestion des secrets.

Bonnes pratiques de sécurité

- ne jamais versionner les vraies clés ;
- conserver le fichier `.env` hors Git ;
- fournir un `.env.example` pour documenter les variables nécessaires ;
- renouveler toute clé ayant été exposée ;
- éviter les URLs codées en dur dans le frontend.

9 Base de données MongoDB

La base MongoDB constitue la couche de persistance du projet. Le dossier technique permet d'identifier les collections principales ainsi que leur rôle métier.

Collections principales et usage

`users` contient les comptes et rôles ; `clients` et `freelancers` portent les profils métier ; `gigs` décrit les services proposés ; `jobs` représente les besoins publiés ; `proposals` matérialise les candidatures ; `orders` suit les commandes de gigs ; `contracts` formalise les accords ; `payments` suit les transactions ; `conversations` et `messages` gèrent la messagerie ; `reviews`, `sprint_plans`, `sprints` et `sprint_reviews` complètent la traçabilité de l'exécution.

Le choix de MongoDB est cohérent avec un domaine métier où plusieurs documents peuvent évoluer de façon souple et où certaines structures, comme les profils, les conversations ou les entités de suivi de sprint, peuvent présenter une certaine variabilité.

10 Flux métiers principaux

Le dossier technique décrit explicitement plusieurs flux métiers. Les intégrer dans le rapport permet de relier l'architecture et le code aux usages réels de la plateforme.

10.1 Inscription et login

Le cycle commence par l'inscription de l'utilisateur comme client ou freelancer. Le backend crée ensuite un document dans `users`, puis un profil dans `clients` ou `freelancers`. Lors de la connexion, un JWT est généré puis stocké côté frontend pour les appels ultérieurs.

10.2 Rechercher et engager un freelancer

Le client explore les talents ou le catalogue, consulte un profil ou un gig, puis publie un job ou commande directement une prestation. Selon le cas, le backend crée un `job`, une `proposal` ou une `order`. Ce flux montre l'articulation entre découverte, sélection et engagement.

10.3 Exécution d'une commande gig

Le client commande un gig ; le freelancer prend la commande en charge ; il prépare et livre le travail ; le client valide ; enfin le paiement est marqué dans le système. Ce flux est principalement supporté par `order_routes.py`, `order_service.py` et `payment_service.py`.

10.4 Travail par contrat et sprints

Dans les projets plus structurés, un contrat est d'abord créé. Un plan de sprint est ensuite défini, les sprints sont exécutés, le freelancer soumet son travail, puis le client valide ou rejette les livrables. Les paiements peuvent être libérés en fonction des validations successives.

10.5 Chat

Une conversation est créée, les messages sont stockés, puis Socket.IO notifie les participants en temps réel. Ce flux montre que la plateforme ne se contente pas d'afficher des données : elle maintient aussi une interaction continue entre les parties.

11 Analyse fonctionnelle

11.1 Fonctionnalités de base côté client

Le client est l'acteur qui exprime un besoin, sélectionne un prestataire et suit l'exécution du service. Les fonctionnalités fondamentales qui lui sont associées sont décrites ci-après.

11.1.1 Déposer un job

La publication d'un job constitue le point de départ d'un projet. Le client renseigne un titre, une description, les compétences attendues, le budget et le délai. Cette information devient ensuite exploitable par les freelancers, qui peuvent formuler une candidature adaptée.

D'un point de vue métier, cette fonctionnalité joue plusieurs rôles : elle formalise le besoin, crée une base de comparaison entre les offres reçues et permet de tracer le lancement du projet dans le système.

11.1.2 Rechercher un profil freelancer

Le moteur de recherche permet au client d'identifier des prestataires selon plusieurs critères tels que le domaine de compétence, les technologies maîtrisées, la disponibilité, la réputation ou la gamme tarifaire. La fiche d'un freelancer peut comprendre sa biographie, son portfolio, ses évaluations, ses gigs et des informations de fiabilité.

Cette fonctionnalité est essentielle pour réduire l'incertitude lors du choix d'un prestataire et améliorer la pertinence du matching entre besoin et compétence.

11.1.3 Communiquer avec un freelancer via message

La messagerie intégrée centralise les échanges au sein de la plateforme. Elle sert à discuter du besoin initial, à clarifier des exigences, à partager des précisions, à suivre l'avancement et à conserver une trace des engagements pris pendant le projet.

D'un point de vue technique, cette fonctionnalité peut s'appuyer sur des échanges persistants en base de données combinés à un canal temps réel pour fluidifier l'expérience utilisateur.

11.1.4 Effectuer le paiement

Le paiement représente l'étape de concrétisation de la relation contractuelle. Une fois une offre validée, le client doit pouvoir confirmer le montant, autoriser la transaction et recevoir un état de paiement. Dans une logique plus avancée, le système peut gérer des paiements par jalons ou par sprint, ce qui améliore la maîtrise du risque pour les deux parties.

11.2 Fonctionnalités de base côté freelancer

Le freelancer est l'acteur qui met en valeur son expertise, répond à des opportunités et réalise les prestations sélectionnées.

11.2.1 Postuler à un job

Le freelancer consulte les besoins publiés et soumet une candidature. Celle-ci peut comporter un message personnalisé, une estimation budgétaire, un délai prévisionnel ainsi que des références liées à des travaux antérieurs. Ce mécanisme favorise la mise en concurrence constructive et aide le client à comparer plusieurs approches.

11.2.2 Publier ses gigs

Le gig correspond à une offre packagée proposée directement par le freelancer. Il présente un service standardisé, un prix, un délai et éventuellement des options complémentaires. Ce modèle simplifie les commandes rapides et offre au freelancer un canal de vente complémentaire à la candidature sur job.

11.2.3 Suivre l'exécution de ses missions

Une fois un projet attribué, le freelancer doit pouvoir consulter les détails du contrat, échanger avec le client, transmettre des livrables, suivre le statut de validation et vérifier l'état des paiements. Cette fonctionnalité garantit une meilleure visibilité sur l'avancement global de la mission.

11.3 Explication détaillée des routes backend

Le dossier technique met en évidence un ensemble de routes organisées par domaines métiers. Dans une architecture Flask fondée sur des blueprints, cette approche permet de regrouper les endpoints selon leur responsabilité fonctionnelle et de mieux contrôler les accès selon le rôle de l'utilisateur.

11.3.1 Routes d'authentification

Les routes d'authentification constituent le point d'entrée du système. Elles sont généralement regroupées sous le préfixe `/api/auth`. Trois endpoints majeurs sont identifiés : `POST /api/auth/register/client`, `POST /api/auth/register/freelancer` et `POST /api/auth/login`.

Leur rôle est respectivement de créer un compte client, de créer un compte freelancer et de vérifier les identifiants lors de la connexion. Après authentification réussie, le backend génère un jeton JWT contenant l'identifiant de l'utilisateur ainsi que son rôle. Ce mécanisme sert ensuite à sécuriser les autres routes de l'application.

11.3.2 Routes liées aux freelancers

Les routes du module `freelancer_routes.py`, exposées sous `/api/freelancer`, couvrent la consultation et la gestion des profils freelancers. Elles permettent de lire un profil, de mettre à jour les informations professionnelles, de modifier le mot de passe, d'accéder au tableau de bord freelancer et d'effectuer des recherches publiques de talents.

Ces routes présentent une double nature : certaines sont publiques, par exemple pour afficher un profil ou lister des freelancers, tandis que d'autres nécessitent un JWT valide et un rôle `freelancer`, notamment pour les opérations de modification de profil ou l'accès au dashboard personnel.

11.3.3 Routes liées aux clients

Les routes gérées par `client_routes.py` sont dédiées à l'espace client. Elles servent principalement à consulter le profil client, à le mettre à jour, à modifier le mot de passe et à afficher le tableau de bord du client.

L'accès à ces routes est protégé par JWT et réservé au rôle `client`. Cette séparation garantit qu'un freelancer ne peut pas accéder directement aux fonctions internes du compte client, ce qui renforce la cohérence métier et la sécurité applicative.

11.3.4 Routes de gestion des gigs et du catalogue

Le dossier distingue deux usages complémentaires. Le premier concerne les routes de `gig_routes.py`, qui permettent au freelancer de créer, modifier, supprimer et consulter ses propres gigs. Le second concerne les routes de catalogue, destinées au client ou au public, pour consulter les services proposés, appliquer des filtres, visualiser le détail d'un gig et déclencher une commande.

Cette distinction entre *gestion interne des gigs* et *catalogue de consultation* traduit une bonne pratique d'architecture. Elle évite de mélanger les opérations d'administration d'une offre avec les parcours d'achat ou d'exploration destinés aux clients.

11.3.5 Routes de jobs et de propositions

Les modules `job_routes.py` et `proposal_routes.py` prennent en charge le cycle de recrutement d'un freelancer à partir d'un besoin publié. Les routes de jobs permettent au client de publier un projet, de consulter ses annonces ou d'en suivre l'état. Les routes de propositions permettent aux freelancers d'envoyer une candidature, puis au client de consulter, comparer et sélectionner les propositions reçues.

Ces routes constituent le cœur du processus de mise en relation. Elles établissent le passage entre une intention exprimée par le client et une collaboration potentielle avec un freelancer.

11.3.6 Routes de commandes

Les routes associées à `order_routes.py` gèrent le cycle de vie d'une commande issue d'un gig. Le dossier décrit une séquence composée des états `pending`, `in_progress`, `submitted` puis `completed`. Le client initie la commande, le freelancer l'accepte et l'exécute, puis le client valide la livraison et le paiement finalise la commande.

Ces routes sont fortement dépendantes du rôle utilisateur. Les opérations réservées au client et celles réservées au freelancer sont distinctes, ce qui évite les ambiguïtés de responsabilité et améliore la fiabilité des transitions d'état.

11.3.7 Routes de contrats, plans de sprint et sprints

Les fichiers `contract_routes.py`, `sprint_plan_routes.py` et `sprint_routes.py` structurent la gestion avancée des projets. Les routes de contrat servent à formaliser l'accord entre les parties. Les routes de plan de sprint servent à préparer le découpage du projet. Enfin, les routes de sprint permettent de suivre l'exécution, la soumission des livrables et la validation des étapes successives.

Ce groupe de routes est particulièrement important dans une logique de pilotage incrémental. Il permet d'encadrer les engagements, d'associer les paiements à des jalons et d'assurer une meilleure traçabilité du projet.

11.3.8 Routes de paiement

Les routes définies dans `payment_routes.py` prennent en charge le financement et le suivi des paiements. Elles servent à enregistrer les transactions, à suivre leur état et à articuler la logique financière avec l'état des commandes, contrats ou sprints.

Dans une plateforme de services, ces routes doivent être soigneusement sécurisées car elles manipulent des opérations sensibles. Leur rôle est central pour instaurer la confiance entre client et freelancer.

11.3.9 Routes de chat et de communication temps réel

Le module `chat.py` ainsi que le composant `chat_socket.py` organisent la messagerie de la plateforme. Les routes HTTP servent à créer des conversations, récupérer la liste des conversations et charger l'historique des messages. Le canal Socket.IO permet quant à lui d'assurer l'envoi et la réception en temps réel.

Cette combinaison entre persistance et temps réel améliore la fluidité des échanges tout en conservant une trace durable des discussions. Le dossier mentionne aussi une validation possible des messages via `pii_service.py`, ce qui ouvre la voie à un contrôle de contenu sensible.

11.3.10 Routes d'administration

Enfin, les routes d'administration regroupées dans `admin_routes.py` permettent à un administrateur de superviser la plateforme. Elles couvrent la consultation des gigs et jobs en attente, l'approbation ou le rejet de certains contenus, la consultation de statistiques et la gestion de l'état des comptes utilisateurs.

Ces routes sont réservées au rôle `admin`. Elles constituent un niveau de contrôle transverse indispensable pour modérer la plateforme, suivre son activité et assurer son bon fonctionnement à grande échelle.

12 Architecture applicative détaillée

12.1 Couche backend

La couche backend assure la cohérence métier de la plateforme. Les principales responsabilités identifiées sont les suivantes :

- gestion de l'authentification et des rôles utilisateur ;
- gestion des profils clients et freelancers ;
- publication des jobs et gestion des candidatures ;
- publication et commande des gigs ;
- gestion des contrats, des paiements et des sprints ;
- gestion de la messagerie et des conversations.

Cette couche s'appuie sur un schéma classique : les routes reçoivent les requêtes, les services exécutent la logique métier, puis les modèles assurent la persistance des données. Une telle organisation améliore la testabilité, la réutilisation du code et la séparation des responsabilités.

12.2 Couche frontend

La couche frontend orchestre l'expérience utilisateur. Elle regroupe les pages d'authentification, les espaces client, les espaces freelancer, les écrans de détail des jobs, les écrans de messagerie, les formulaires de gestion des gigs et les pages de suivi des projets.

L'utilisation d'Angular favorise la structuration en composants réutilisables, tandis qu'Ionic facilite le déploiement sur des interfaces mobiles et web avec une cohérence visuelle et fonctionnelle.

12.3 Données manipulées par le système

Les principales entités de données nécessaires à la solution sont :

- les utilisateurs et leurs rôles ;
- les profils clients et freelancers ;
- les jobs ;
- les propositions ou candidatures ;
- les gigs et commandes ;
- les conversations et messages ;
- les paiements ;
- les contrats, plans de sprint et sprints.

Ces entités couvrent l'ensemble du cycle de vie métier, depuis l'inscription d'un utilisateur jusqu'à la clôture d'une prestation.

13 Gestion des projets en sprints

13.1 Justification du découpage en sprints

Dans une plateforme de freelancing orientée vers la réalisation de missions numériques, le découpage en sprints présente un intérêt majeur. Il permet de structurer le projet en étapes courtes, de rendre l'avancement visible, de mieux prioriser les tâches et de réduire les risques de dérive fonctionnelle. Cette approche est particulièrement pertinente lorsque le besoin initial doit être clarifié progressivement ou validé au travers de livrables intermédiaires.

13.2 Cycle de vie d'un sprint fonctionnel

Chaque sprint peut suivre la séquence suivante :

1. définition des objectifs et du périmètre ;
2. estimation de la charge, du délai et du coût ;
3. exécution par le freelancer ;
4. dépôt du livrable ;
5. revue par le client ;

6. validation, demande de correction ou passage au sprint suivant.

13.3 Bénéfices attendus

L'approche par sprints apporte plusieurs avantages :

- meilleure visibilité sur l'état réel du projet ;
- validation incrémentale des résultats ;
- amélioration de la relation client–freelancer ;
- réduction du risque lié au paiement global en fin de mission ;
- meilleure traçabilité des décisions et des livrables.

14 Planification du développement en sprints

Afin de produire la plateforme de manière progressive et maîtrisée, le développement peut être structuré en six sprints de réalisation.

Sprint	Objectif principal	Livrables attendus
Sprint 1	Cadrage et mise en place du socle	Initialisation des projets Ionic/Angular et Flask, structuration du dépôt, configuration des environnements, premières routes API, base d'authentification et mise en place de la base de données.
Sprint 2	Comptes et profils utilisateurs	Inscription et connexion des clients et freelancers, gestion des rôles, création et mise à jour des profils, sécurisation des parcours et des accès.
Sprint 3	Gestion des jobs, recherche et gigs	Publication des jobs, consultation des offres, recherche de freelancers, affichage des profils, création et administration des gigs, mise en place des tableaux de bord essentiels.
Sprint 4	Candidatures et messagerie	Soumission des propositions, consultation des candidatures, gestion des conversations, envoi et réception de messages, notifications de base et conservation de l'historique.
Sprint 5	Paiement et contractualisation	Validation des offres, génération des commandes ou contrats, suivi des paiements, états de transaction, articulation entre exécution de mission et règlement.

Sprint	Objectif principal	Livrables attendus
Sprint 6	Gestion par sprints, tests et stabilisation	Découpage d'un projet en sprints, suivi d'avancement, validation par étape, optimisation ergonomique, correction des anomalies, tests finaux et préparation au déploiement.

15 Exigences non fonctionnelles

Pour répondre aux attentes d'un système professionnel et durable, la solution doit respecter plusieurs exigences transversales.

15.1 Sécurité

La sécurité constitue un enjeu majeur. Elle implique la protection des endpoints sensibles, la validation des données saisies, la gestion sécurisée des tokens d'authentification, le contrôle des rôles et la sécurisation des opérations de paiement.

15.2 Performance

Le système doit fournir des temps de réponse satisfaisants, en particulier sur les recherches de profils, la consultation des jobs, l'affichage des conversations et les tableaux de bord. Une bonne organisation des appels API et des accès aux données est donc essentielle.

15.3 Maintenabilité et évolutivité

La séparation entre interface, services et logique métier favorise la maintenabilité. L'architecture modulaire permet également d'ajouter ultérieurement de nouvelles fonctionnalités, telles que des évaluations avancées, des outils d'analyse, un système de recommandation ou une gestion administrative plus poussée.

15.4 Ergonomie et expérience utilisateur

Le recours à Ionic vise à garantir une interface lisible et homogène sur plusieurs terminaux. L'ergonomie doit privilégier des parcours simples, un vocabulaire explicite, des formulaires clairs et un accès direct aux fonctionnalités essentielles.

16 Discussion et appréciation technique

L'analyse du dossier technique et des captures de structure met en évidence une architecture cohérente pour une application de type marketplace de freelancing. La séparation entre les modules du backend, la granularité des services et la richesse des pages frontend montrent une volonté de structurer le produit autour de domaines métiers identifiés.

Cette architecture reste pertinente pour un projet de marketplace de freelancing : elle sépare clairement les responsabilités, couvre les domaines métiers essentiels et offre une base technique exploitable par un développeur qui souhaite comprendre, maintenir ou faire évoluer la plateforme.

17 Conclusion générale

Freelancer Hub constitue une solution technique pertinente pour répondre aux besoins d'une plateforme de mise en relation entre clients et freelancers. Le choix d'un frontend **Ionic/Angular** et d'un backend **Flask** permet de combiner souplesse de développement, clarté architecturale et compatibilité avec des usages web et mobiles.

L'étude met en évidence un noyau fonctionnel solide : dépôt de jobs, recherche de profils, messagerie, candidatures, gestion des gigs, paiement et pilotage des missions par sprints. L'intégration des captures d'arborescence confirme la structuration des composants backend et frontend, ce qui renforce la crédibilité technique de la solution.

Dans une perspective académique comme professionnelle, ce projet présente donc une base sérieuse pour la réalisation d'une plateforme évolutive, maintenable et adaptée aux exigences d'un environnement numérique de services freelance.